

TEMA 01 / PROGRAMACIÓN DE SERVICIOS Y PROCESOS

Conceptos básicos de procesos y conurrencia

Sistemas operativos, planificación y
creación de procesos con Java

Fundamentos

Gestión de procesos

Java

Comunicación

APUNTES DE CLASE

Teoría · esquemas · código comentado · actividades

CPU / JVM

GUÍA DE CONTENIDOS

Índice del tema

Siete bloques de estudio, desde el funcionamiento del sistema operativo hasta la comunicación entre procesos en Java.

01 Conceptos básicos

Programa y proceso • ejecutable • sistema operativo

02 Funcionamiento del sistema operativo

Kernel • interrupciones • llamadas al sistema

03 Los procesos

Estados • colas • planificación • cambio de contexto

04 Gestión de procesos

Jerarquía padre-hijo • PID • operaciones básicas

05 Programación concurrente

Concurrencia • multiprogramación • paralelismo

06 Creación de procesos con Java

ProcessBuilder • Process • espera y terminación

07 Comunicación entre procesos

stdin • stdout • stderr • flujos de Java

Al finalizar el tema

Podrás distinguir programas de procesos, explicar el ciclo de vida de un proceso y crear, supervisar y comunicar procesos externos desde una aplicación Java.

Recursos de consulta

Documentación oficial de Java: [ProcessBuilder](#) y [Process](#). Referencia de comandos Linux: [ps](#).

BLOQUE 01

1. Conceptos básicos

Un mismo programa puede dar lugar a varios procesos independientes.

1.1 Programa, ejecutable y proceso

Un **programa** es un conjunto de instrucciones y datos organizados para realizar una tarea. Cuando el sistema pone en marcha ese programa, se crea un **proceso**: una instancia de ejecución con recursos y estado propios.

Un **ejecutable** es un archivo que contiene instrucciones e información necesaria para poner un programa en ejecución en un entorno concreto. En Java, por ejemplo, los archivos `.class` contienen bytecode que ejecuta la máquina virtual (JVM): no son ejecutables nativos del sistema operativo.

Concepto	Qué representa	Ejemplo
Programa	Instrucciones y datos disponibles para realizar una tarea.	Un editor de texto instalado.
Ejecutable	Archivo o formato que el entorno de ejecución puede cargar.	Un binario del sistema o bytecode Java.
Proceso	Instancia de un programa que se está ejecutando o esperando para hacerlo.	Dos ventanas abiertas del editor pueden corresponder a dos procesos.

Idea clave

Dos procesos pueden ejecutar el mismo programa y tener distinto PID, memoria, permisos, archivos abiertos y estado de ejecución.

1.2 Información asociada a un proceso

- **Identificadores:** PID (identificador de proceso) y, cuando procede, PPID (identificador del padre).
- **Contexto de CPU:** contador de programa, registros y datos necesarios para continuar la ejecución.
- **Espacio de direcciones:** memoria asignada para código, pila y datos.
- **Recursos y seguridad:** archivos abiertos, credenciales, permisos y otros recursos gestionados por el sistema operativo.
- **Estado:** listo, en ejecución, bloqueado u otro estado del modelo de procesos.

1.3 Procesos en segundo plano y servicios

Un **demonio** (*daemon*) es un proceso de servicio que suele funcionar en segundo plano, sin una interfaz interactiva directa. Puede permanecer esperando eventos o peticiones: no tiene por qué consumir CPU continuamente. En sistemas Windows es habitual hablar de **servicios**.

1.4 Funciones del sistema operativo

El **sistema operativo (SO)** coordina el hardware y ofrece servicios a los programas. Crea y finaliza procesos, distribuye tiempo de CPU, gestiona memoria, archivos, dispositivos y mecanismos de seguridad.



Las aplicaciones solicitan servicios mediante interfaces del sistema operativo; el kernel realiza las operaciones privilegiadas sobre los recursos del equipo.

BLOQUE 02

2. Funcionamiento del sistema operativo

El kernel conecta las necesidades de los programas con los recursos físicos del equipo.

2.1 Kernel e interrupciones

El **kernel** o núcleo es el componente del sistema operativo que administra procesos, memoria y dispositivos, y proporciona mecanismos de protección y comunicación. Puede ejecutar código por interrupciones, excepciones, llamadas al sistema y otras actividades internas.

Una **interrupción** es una señal que hace que el procesador atienda un suceso, como la finalización de una operación de entrada/salida. Una **excepción** se produce durante la ejecución de una instrucción, por ejemplo ante una operación no permitida. Su tratamiento depende del hardware y del sistema operativo.

Interrupciones y reanudación

La CPU guarda la información necesaria para atender el evento. Tras la rutina correspondiente, puede reanudarse el mismo proceso o ejecutarse otro si el planificador toma esa decisión.

2.2 Modo usuario y modo kernel

Modo usuario	Modo kernel
Ejecuta normalmente el código de las aplicaciones con permisos limitados.	Permite ejecutar instrucciones privilegiadas y acceder de forma controlada a recursos protegidos.
No puede manipular libremente memoria ajena ni dispositivos.	Comprueba permisos y gestiona solicitudes del sistema.

2.3 Llamadas al sistema

Una **llamada al sistema** (*system call*) es una solicitud de una aplicación al kernel para realizar una operación controlada: abrir un archivo, crear un proceso o intercambiar datos con un dispositivo. El mecanismo de entrada al kernel y la transferencia de control dependen de la arquitectura.

Una aplicación Java suele acceder a estos servicios mediante las bibliotecas de Java y la JVM, que utilizan los mecanismos adecuados del sistema operativo.

BLOQUE 03

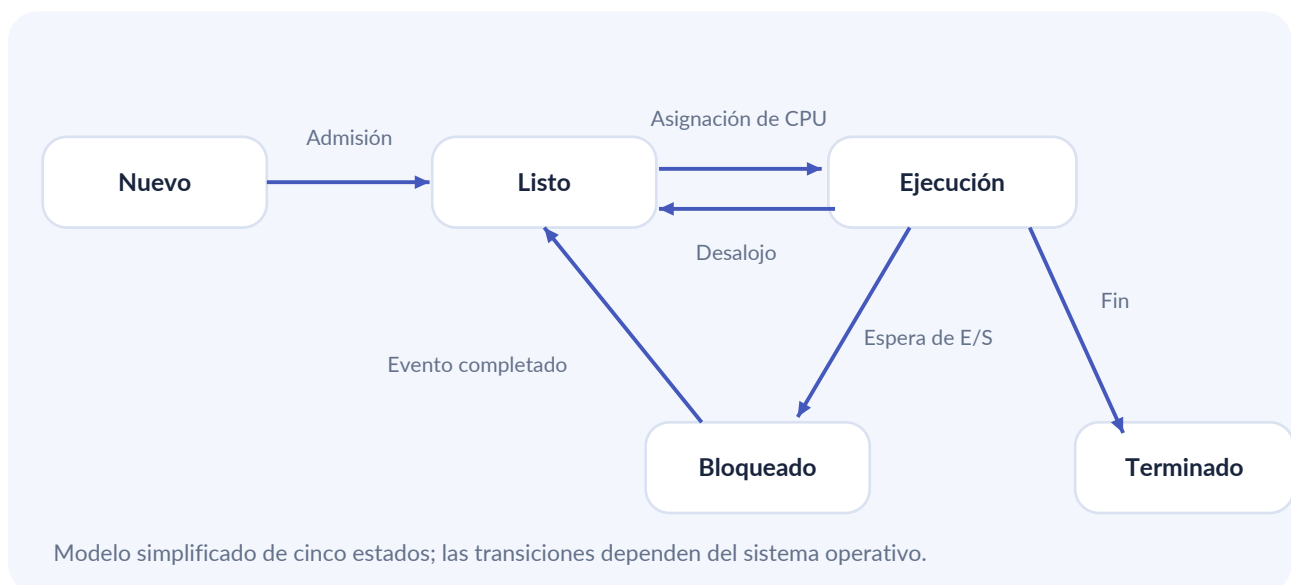
3. Los procesos

El sistema operativo gestiona el ciclo de vida de los procesos y decide cuál puede utilizar la CPU.

3.1 Estados de un proceso

Para estudiar su funcionamiento se emplea un **modelo simplificado de cinco estados**. Un sistema operativo real puede utilizar estados y colas adicionales.

Estado	Significado
Nuevo	El sistema está creando el proceso y preparando sus recursos.
Listo	Puede ejecutarse, pero aún no tiene asignada una CPU.
En ejecución	Está utilizando una CPU o un núcleo en ese instante.
Bloqueado / espera	Espera un evento, como la finalización de una operación de E/S o un mecanismo de sincronización.
Terminado	Ha finalizado la ejecución; el sistema libera sus recursos cuando corresponde.



Transición importante

Cuando termina una operación de E/S, el proceso pasa normalmente de **bloqueado** a **listo**. No vuelve necesariamente a ejecutarse en ese mismo instante.

3.2 Colas de procesos

Para organizar el trabajo, el sistema operativo mantiene estructuras con procesos listos para ejecutarse y con procesos que esperan eventos o recursos. Pueden existir colas asociadas a dispositivos, niveles de prioridad o políticas de planificación.

3.3 Planificación a corto y largo plazo

Planificador	Decisión principal
Corto plazo	Escoge qué proceso o hilo listo obtiene una CPU. Se activa con mucha frecuencia y debe ser rápido.
Largo plazo	En el modelo clásico, decide qué trabajos nuevos se admiten al sistema y regula el grado de multiprogramación. No todos los sistemas actuales lo implementan como componente separado.

3.4 Cambio de contexto

Un **cambio de contexto** ocurre cuando el sistema guarda la información necesaria para suspender un proceso o hilo y restaura la de otro. Incluye datos de la CPU y, según el caso, información de memoria y ejecución. Este trabajo supone una sobrecarga: mientras se efectúa el cambio no se están ejecutando instrucciones útiles del proceso de usuario.

La planificación permite repartir el procesador entre distintas tareas, pero un número excesivo de cambios de contexto también puede perjudicar el rendimiento.

BLOQUE 04

4. Gestión de procesos

La creación de procesos establece relaciones entre instancias de ejecución y permite supervisar su actividad.

4.1 PID y relaciones padre-hijo

El sistema operativo asigna a cada proceso un **PID** para identificarlo mientras existe. En muchos sistemas también puede consultarse el **PPID**, que identifica el proceso padre. En sistemas tipo Unix, un proceso suele crear otros procesos mediante servicios del sistema y se forma una jerarquía.

Un proceso hijo tiene su propio espacio de direcciones; aun así, puede heredar determinados recursos o establecer canales de comunicación con el padre, según el sistema operativo y la forma de creación.

Cuando finaliza el padre

No debe suponerse que sus hijos terminan siempre en cascada. En sistemas tipo Unix pueden continuar y ser adoptados por otro proceso; el comportamiento exacto depende del sistema y de cómo se hayan configurado los procesos.

4.2 Operaciones habituales

Operación	Función
Crear	Pedir al sistema que inicie un nuevo proceso. En Java, <code>ProcessBuilder.start()</code> .
Esperar	Suspender la ejecución del llamante hasta que finalice el proceso observado, por ejemplo mediante <code>waitFor()</code> .
Finalizar	Solicitar el fin de un proceso. En Java, <code>destroy()</code> ; en algunos casos puede ser necesario <code>destroyForcibly()</code> .

4.3 Actividad de observación en Linux

En un entorno Linux de prácticas, abre una terminal y utiliza estos comandos para observar procesos sin alterar el funcionamiento del sistema:

Comandos de observación

```
ps -ef           # Muestra los procesos
pstree -p        # Muestra el árbol y los PID
sleep 120 &      # Inicia una espera en segundo plano
echo $!          # PID del último proceso en segundo plano
kill PID         # Sustituye PID por el valor anterior
```

BLOQUE 05

5. Programación concurrente

Varias tareas pueden avanzar durante un mismo intervalo; eso no significa necesariamente que se ejecuten a la vez.

5.1 Concurrencia y paralelismo

La **concurrencia** permite que varias tareas progresen durante un mismo intervalo de tiempo, alternando sus avances o ejecutándose simultáneamente. El **paralelismo** implica ejecución real al mismo tiempo en distintos núcleos o recursos de cómputo.

Concepto	Qué ocurre	Ejemplo
Concurrencia	Varias tareas avanzan y comparten tiempos de ejecución.	Un único núcleo alterna entre dos procesos.
Paralelismo	Dos o más tareas se ejecutan en el mismo instante.	Dos núcleos realizan cálculos diferentes a la vez.
Multiprogramación	El SO mantiene varios trabajos o procesos y administra su acceso al procesador.	Mientras uno espera una lectura de disco, otro utiliza la CPU.

Concurrencia ≠ paralelismo

Un equipo con un solo núcleo puede ejecutar programas concurrentes sin que sus instrucciones se ejecuten de forma simultánea. Con varios núcleos, la concurrencia puede combinarse con paralelismo.

5.2 Multiprogramación y aprovechamiento de la CPU

La multiprogramación permite mantener diversos programas o procesos disponibles y aprovechar el tiempo en que algunos esperan operaciones de entrada/salida. El planificador selecciona qué tarea puede usar la CPU según la política del sistema operativo.

No garantiza que **cada tarea** termine antes: los resultados dependen de la carga de trabajo, la cantidad de CPU, los tiempos de espera y los costes de coordinación.

5.3 Paralelismo y memoria

En un equipo multinúcleo, varios hilos de un mismo proceso pueden ejecutar instrucciones a la vez y compartir memoria. Los procesos distintos suelen tener espacios de direcciones separados y necesitan mecanismos específicos para comunicarse. En el **cálculo distribuido**, varios ordenadores colaboran mediante mensajes o redes; por ejemplo, con herramientas como MPI.

BLOQUE 06

6. Creación de procesos con Java

Java permite iniciar procesos del sistema operativo mediante `ProcessBuilder` y supervisarlos desde objetos `Process`.

6.1 `ProcessBuilder` y `Process`

`ProcessBuilder` configura y lanza un proceso externo. Sus opciones incluyen `command(...)` (programa y argumentos), `directory(...)` (directorio de trabajo), `environment()` (entorno) y `redirectErrorStream(...)` (tratamiento de la salida de error).

Al invocar `start()`, Java devuelve un objeto `Process` para consultar su estado, comunicarse con él, esperar su finalización o solicitar que termine.

Método	Uso
<code>start()</code>	Inicia el proceso; puede lanzar <code>IOException</code> si falla el arranque.
<code>waitFor()</code>	Espera hasta que termina y devuelve su código de salida; puede lanzar <code>InterruptedException</code> .
<code>isAlive()</code>	Indica si el proceso continúa activo.
<code>exitValue()</code>	Devuelve su código de salida si ya ha finalizado; en caso contrario lanza una excepción.
<code>destroy()</code>	Solicita terminar el proceso; la respuesta y el tiempo de finalización dependen de la plataforma.

6.2 Ejemplo: iniciar un programa externo y obtener su resultado

El siguiente programa ejecuta `java -version` usando el ejecutable de la instalación de Java actual. Combina salida estándar y salida de error, lee el resultado y espera al proceso.

Ejemplo 1 • VersionJava.java

```
import java.io.*;
import java.nio.charset.StandardCharsets;

public class VersionJava {
    public static void main(String[] args) throws Exception {
        String java = System.getProperty("java.home")
            + File.separator + "bin" + File.separator + "java";
        ProcessBuilder pb = new ProcessBuilder(java, "-version");
        pb.redirectErrorStream(true);
        Process proceso = pb.start();

        try (BufferedReader salida = new BufferedReader(
            new InputStreamReader(proceso.getInputStream(),
                StandardCharsets.UTF_8))) {
            String linea;
            while ((linea = salida.readLine()) != null) {
                System.out.println(linea);
            }
        }
        int codigo = proceso.waitFor();
        System.out.println("Codigo de salida: " + codigo);
    }
}
```

Lectura del ejemplo

java -version suele escribir en la salida de error. Con `redirectErrorStream(true)`, ambas salidas se combinan para leerlas desde `getInputStream()`.

6.3 Terminación y códigos de salida

El método `waitFor()` espera a que el proceso finalice. Por convenio, muchos programas utilizan `0` para indicar éxito y un valor distinto de cero para señalar una incidencia; el significado concreto lo establece cada programa.

Si se necesita solicitar el fin de un proceso, puede utilizarse `destroy()`. Si no termina, `destroyForcibly()` intenta forzarlo. Ninguno de estos métodos implica que el proceso haya desaparecido instantáneamente: comprueba su estado o espera a su finalización.

6.4 Errores de arranque

La creación depende del sistema operativo: pueden producirse errores por ruta inexistente, ausencia de permisos, formato incompatible o recursos insuficientes. Captura `IOException` cuando sea importante gestionar el fallo y trata `InterruptedException` cuando esperes al proceso.

BLOQUE 07

7. Comunicación entre procesos

Los procesos pueden intercambiar datos mediante flujos de entrada, salida y error.

7.1 Los tres flujos estándar

Flujo del hijo	Función	Acceso desde Java (padre)
stdin	Datos que recibe el proceso hijo.	<code>getOutputStream()</code> : escribir hacia el hijo.
stdout	Datos ordinarios que produce el hijo.	<code>getInputStream()</code> : leer la salida del hijo.
stderr	Mensajes de error o diagnóstico del hijo.	<code>getErrorStream()</code> : leer sus errores.

Los nombres `getOutputStream()` y `getInputStream()` se entienden **desde la perspectiva del proceso padre**: el padre escribe en la entrada del hijo y lee la salida del hijo.

Salida y error

Por defecto, `redirectErrorStream(false)` conserva `stdout` y `stderr` por separado. Con `redirectErrorStream(true)` se combinan; así basta leer un único flujo de salida.

7.2 Ejemplo de comunicación bidireccional

Guarda las siguientes clases en dos archivos del mismo directorio. `PadreProceso` crea `HijoSaludo`, le envía un nombre, recoge su respuesta y espera su finalización.

Ejemplo 2 · `HijoSaludo.java`

```
import java.io.*;
import java.nio.charset.StandardCharsets;

public class HijoSaludo {
    public static void main(String[] args) throws IOException {
        BufferedReader entrada = new BufferedReader(
            new InputStreamReader(System.in,
                StandardCharsets.UTF_8));
        String nombre = entrada.readLine();
        System.out.println("Hola, " + nombre + ".");
    }
}
```

Ejemplo 3 • PadreProceso.java

```
import java.io.*;
import java.nio.charset.StandardCharsets;

public class PadreProceso {
    public static void main(String[] args) throws Exception {
        ProcessBuilder pb = new ProcessBuilder(
            "java", "-cp", System.getProperty("java.class.path"),
            "HijoSaludo");
        pb.redirectErrorStream(true);
        Process hijo = pb.start();

        try (PrintWriter entrada = new PrintWriter(
            new OutputStreamWriter(hijo.getOutputStream(),
                StandardCharsets.UTF_8), true)) {
            entrada.println("Ada");
        }
        try (BufferedReader salida = new BufferedReader(
            new InputStreamReader(hijo.getInputStream(),
                StandardCharsets.UTF_8))) {
            String linea;
            while ((linea = salida.readLine()) != null) {
                System.out.println("Hijo: " + linea);
            }
        }
        System.out.println("Codigo de salida: " + hijo.waitFor());
    }
}
```

Compilación y ejecución • terminal con JDK instalado

```
javac HijoSaludo.java PadreProceso.java
java PadreProceso
```

Para ejecutar el ejemplo, los comandos `java` y `javac` deben estar disponibles en la terminal. Si utilizas un IDE, compila ambas clases y ejecuta `PadreProceso` con el directorio de salida en el classpath.

7.3 Evitar bloqueos por uso de buffers

Los flujos tienen capacidad limitada. Si el hijo genera muchos datos y el padre espera su finalización **sin consumir la salida**, el hijo puede quedar bloqueado intentando escribir. Para evitarlo, lee los flujos mientras el proceso sigue en marcha, redirígelos a archivos o combínalos cuando proceda.

7.4 Ejercicios prácticos

- 01** Modifica `PadreProceso` para enviar otro nombre y describe qué flujo usa el padre para mandar el dato.
- 02** Haz que `HijoSaludo` escriba un error con `System.err.println(...)`. Separa los flujos con `redirectErrorStream(false)` y lee `getErrorStream()`.
- 03** Explica por qué, al manejar una salida extensa, puede ser problemático invocar `waitFor()` antes de leer los flujos.
- 04** Elabora un esquema del ciclo de vida de un proceso con los cinco estados y describe dos transiciones provocadas por entrada/salida.